2024-2025 学年秋季学期

《计算机网络》

评分：_____

学　　院：计算机科学与工程学院
专　　业：计算机科学与技术
班　　级：计算机 2204 班
姓　　名：李俊洁
学　　号：20225443
任课教师：刘思宇、贾杰

1 引言

TCP 三次握手是 TCP/IP 协议栈中最重要的概念之一，它用于在客户端和服务器之间建立可靠的连接。本次实验旨在利用 Wireshark 抓包分析 TCP 三次握手的过程，观察数据包中不同协议字段的变化，并分析具体的 TCP 连接建立情况。通过结合 Python 编程进行抓包和数据分析，进一步理解 TCP 连接的建立过程及其重要性。

本文档详细描述了使用 Wireshark 和 Python 工具捕获 TCP 握手过程中的数据包，并使用 PyShark 对数据包进行过滤和分析的过程。

2 实验描述

本节详细介绍了本次实验的目标和实现方法，包括抓包工具的配置、数据分析过程及各部分的具体功能。

2.1 抓包与分析工具

本实验的主要工具包括：

- **tcpdump**：用于抓取网络数据包，是一个命令行工具，能够灵活地进行过滤和抓包操作。
- **PyShark**：一个用于分析.pcap 文件的 Python 库，是 tshark 的 Python 接口，可以实现对抓到的数据包进行详细的解析。
- **Wireshark**：用于抓包和分析的网络协议分析工具，通过图形化界面展示数据包的详细信息，帮助理解协议的工作过程。

2.2 系统实现

实验过程分为以下几个主要部分：

- 启动抓包工具，捕获 TCP 数据包。
- 通过 Python 程序自动访问指定的网站，以触发 TCP 三次握手。
- 使用 PyShark 对抓取到的.pcap 文件进行解析，分析 TCP 三次握手的数据包信息。

2.3 实验流程

1. **启动抓包工具**

使用 Python 的 `subprocess` 模块启动 `tcpdump` 进行抓包。抓包工具被配置为只捕获 TCP 包，以减少抓包文件的大小并聚焦于 TCP 三次握手的分析。

2. **自动访问网站**

通过 Python 程序中的 `requests` 模块，自动访问指定的目标网站，以触发 TCP 三次握手过程。这一步模拟了客户端与服务器之间建立连接的过程。

3. **数据包分析**

抓包完成后，使用 `PyShark` 对生成的.pcap 文件进行解析。实验重点是识别三次握手过程中的三个关键数据包：SYN、SYN-ACK、ACK 包，并记录这些数据包中的关键信息，例如源和目的 IP 地址、源和目的端口、序列号等。

以下是部分 Python 实现的代码片段：

```python
import requests
import subprocess
import time
import os
import pyshark
import logging

# 设置抓包文件路径
pcap_file = 'tcp_handshake.pcap'
log_file = 'packet_capture.log'  # 日志文件路径

# 配置 logging 模块，输出信息到控制台和文件
logging.basicConfig(
    level=logging.INFO,  # 设置日志级别
    format='%(asctime)s - %(levelname)s - %(message)s',  # 设置日志格式
    handlers=[
        logging.FileHandler(log_file),  # 输出到文件
        logging.StreamHandler()  # 输出到控制台
    ]
)

# 获取 logger 实例
logger = logging.getLogger()

# 1. 启动 tcpdump 抓包
def start_tcpdump():
    logger.info("Starting tcpdump to capture packets...")
    # 使用 subprocess 启动 tcpdump 命令
    # -i eth0 指定网卡（替换为实际网卡名，如 eth0 或 wlan0）
    # tcp 过滤器只抓取 TCP 数据包
    # -w 指定抓包文件名
    command = ['sudo', 'tcpdump', '-i', 'eth0', 'tcp', '-w', pcap_file]
    tcpdump_process = subprocess.Popen(command, stdout=subprocess.PIPE, stderr=
        subprocess.PIPE)
    return tcpdump_process

# 2. 自动访问指定网站
def visit_website(url):
    try:
        logger.info(f"Visiting website: {url}")
        response = requests.get(url)  # 发起 GET 请求
        logger.info(f"Website {url} responded with status code {response.
            status_code}")
    except requests.exceptions.RequestException as e:
        logger.error(f"Error accessing website: {e}")

# 3. 等待抓包完成并进行分析
def analyze_pcap():
    # 等待文件创建，直到文件存在并且大小大于0
    while not os.path.exists(pcap_file) or os.path.getsize(pcap_file) == 0:
        logger.info(f"Waiting for packet capture file to be created: {pcap_file}
            ")
        time.sleep(1)

    logger.info(f"Analyzing the captured packets from {pcap_file}...")

    # 使用 PyShark 分析抓到的 .pcap 文件
    cap = pyshark.FileCapture(pcap_file)

    # 打印和分析每个数据包
```

```python
    for packet in cap:
        if 'TCP' in packet:
            logger.info(f"Packet Info: {packet}")

            # 识别 SYN 包（客户端发起连接）
            if packet.tcp.flags_syn == '1' and packet.tcp.flags_ack == '0':
                logger.info(f"SYN Packet: {packet}")
                logger.info(f"Source IP: {packet.ip.src}, Destination IP: {packet.ip.dst}")
                logger.info(f"Source Port: {packet.tcp.srcport}, Destination Port: {packet.tcp.dstport}")
                logger.info(f"Sequence Number: {packet.tcp.seq}")

            # 识别 SYN-ACK 包（服务器响应连接请求）
            elif packet.tcp.flags_syn == '1' and packet.tcp.flags_ack == '1':
                logger.info(f"SYN-ACK Packet: {packet}")
                logger.info(f"Source IP: {packet.ip.src}, Destination IP: {packet.ip.dst}")
                logger.info(f"Source Port: {packet.tcp.srcport}, Destination Port: {packet.tcp.dstport}")
                logger.info(f"Sequence Number: {packet.tcp.seq}, Acknowledgment Number: {packet.tcp.ack}")

            # 识别 ACK 包（客户端确认连接建立）
            elif packet.tcp.flags_syn == '0' and packet.tcp.flags_ack == '1':
                logger.info(f"ACK Packet: {packet}")
                logger.info(f"Source IP: {packet.ip.src}, Destination IP: {packet.ip.dst}")
                logger.info(f"Source Port: {packet.tcp.srcport}, Destination Port: {packet.tcp.dstport}")
                logger.info(f"Sequence Number: {packet.tcp.seq}, Acknowledgment Number: {packet.tcp.ack}")

if __name__ == '__main__':
    # 启动 tcpdump 抓包
    tcpdump_process = start_tcpdump()
    # 等待 tcpdump 启动并捕获部分数据
    time.sleep(2)  # 等待抓包工具启动
    # 自动访问网站来触发 TCP 三次握手
    website_url = input("输入要访问的网站:（http://example.com）")
    visit_website(website_url)
    # 给网站请求一些时间以完成三次握手
    time.sleep(5)  # 等待 5 秒，确保抓到所有数据包
    # 停止 tcpdump 抓包
    tcpdump_process.terminate()
    # 分析抓到的数据包
    analyze_pcap()
import requests
import subprocess
import time
import os
import pyshark
import logging

# 设置抓包文件路径
pcap_file = 'tcp_handshake.pcap'
log_file = 'packet_capture.log'  # 日志文件路径

# 配置 logging 模块，输出信息到控制台和文件
logging.basicConfig(
```

```python
    level=logging.INFO,  # 设置日志级别
    format='%(asctime)s - %(levelname)s - %(message)s',  # 设置日志格式
    handlers=[
        logging.FileHandler(log_file),  # 输出到文件
        logging.StreamHandler()  # 输出到控制台
    ]
)

# 获取 logger 实例
logger = logging.getLogger()

# 1. 启动 tcpdump 抓包
def start_tcpdump():
    logger.info("Starting tcpdump to capture packets...")
    # 使用 subprocess 启动 tcpdump 命令
    # -i eth0 指定网卡（替换为实际网卡名，如 eth0 或 wlan0）
    # tcp 过滤器只抓取 TCP 数据包
    # -w 指定抓包文件名
    command = ['sudo', 'tcpdump', '-i', 'eth0', 'tcp', '-w', pcap_file]
    tcpdump_process = subprocess.Popen(command, stdout=subprocess.PIPE, stderr=
        subprocess.PIPE)
    return tcpdump_process

# 2. 自动访问指定网站
def visit_website(url):
    try:
        logger.info(f"Visiting website: {url}")
        response = requests.get(url)  # 发起 GET 请求
        logger.info(f"Website {url} responded with status code {response.
            status_code}")
    except requests.exceptions.RequestException as e:
        logger.error(f"Error accessing website: {e}")

# 3. 等待抓包完成并进行分析
def analyze_pcap():
    # 等待文件创建，直到文件存在并且大小大于0
    while not os.path.exists(pcap_file) or os.path.getsize(pcap_file) == 0:
        logger.info(f"Waiting for packet capture file to be created: {pcap_file}
            ")
        time.sleep(1)

    logger.info(f"Analyzing the captured packets from {pcap_file}...")

    # 使用 PyShark 分析抓到的 .pcap 文件
    cap = pyshark.FileCapture(pcap_file)

    # 打印和分析每个数据包
    for packet in cap:
        if 'TCP' in packet:
            logger.info(f"Packet Info: {packet}")

            # 识别 SYN 包（客户端发起连接）
            if packet.tcp.flags_syn == '1' and packet.tcp.flags_ack == '0':
                logger.info(f"SYN Packet: {packet}")
                logger.info(f"Source IP: {packet.ip.src}, Destination IP: {
                    packet.ip.dst}")
                logger.info(f"Source Port: {packet.tcp.srcport}, Destination
                    Port: {packet.tcp.dstport}")
                logger.info(f"Sequence Number: {packet.tcp.seq}")
```

```python
            # 识别 SYN-ACK 包（服务器响应连接请求）
            elif packet.tcp.flags_syn == '1' and packet.tcp.flags_ack == '1':
                logger.info(f"SYN-ACK Packet: {packet}")
                logger.info(f"Source IP: {packet.ip.src}, Destination IP: {packet.ip.dst}")
                logger.info(f"Source Port: {packet.tcp.srcport}, Destination Port: {packet.tcp.dstport}")
                logger.info(f"Sequence Number: {packet.tcp.seq}, Acknowledgment Number: {packet.tcp.ack}")

            # 识别 ACK 包（客户端确认连接建立）
            elif packet.tcp.flags_syn == '0' and packet.tcp.flags_ack == '1':
                logger.info(f"ACK Packet: {packet}")
                logger.info(f"Source IP: {packet.ip.src}, Destination IP: {packet.ip.dst}")
                logger.info(f"Source Port: {packet.tcp.srcport}, Destination Port: {packet.tcp.dstport}")
                logger.info(f"Sequence Number: {packet.tcp.seq}, Acknowledgment Number: {packet.tcp.ack}")

if __name__ == '__main__':
    # 启动 tcpdump 抓包
    tcpdump_process = start_tcpdump()
    # 等待 tcpdump 启动并捕获部分数据
    time.sleep(2)  # 等待抓包工具启动
    # 自动访问网站来触发 TCP 三次握手
    website_url = input("输入要访问的网站：（http://example.com）")
    visit_website(website_url)
    # 给网站请求一些时间以完成三次握手
    time.sleep(5)  # 等待 5 秒，确保抓到所有数据包
    # 停止 tcpdump 抓包
    tcpdump_process.terminate()
    # 分析抓到的数据包
    analyze_pcap()
```

Listing 1: 抓包与分析的代码片段

3 方法

3.1 使用的技术和库

本实验的实现涉及到以下工具和库：

- `subprocess`：用于启动外部程序（如 `tcpdump`）以实现抓包。
- `requests`：用于模拟客户端发起 HTTP 请求，触发 TCP 三次握手。
- `pyshark`：用于分析抓取到的网络数据包。
- `logging`：用于记录实验过程中的关键信息。

3.2 工作流程

实验的工作流程可以总结如下：

1. 使用 `tcpdump` 抓包工具开始捕获 TCP 数据包。
2. 自动访问目标网站，触发 TCP 连接。

3. 使用 PyShark 解析抓取到的.pcap 文件，分析 TCP 三次握手过程中的数据包。

4 使用场景

本实验的目标是对计算机网络和 TCP 协议工作机制感兴趣的学生和开发者。通过本次实验，可以学习到：

- 如何使用抓包工具捕获和分析 TCP 数据包。
- TCP 三次握手的过程及各个阶段的数据包细节。
- Python 编程在抓包与分析过程中的应用。

4.1 使用场景

- 学习网络协议，理解 TCP 三次握手的详细过程。
- 使用抓包和数据分析技术，排查网络通信中的问题。
- 通过 Python 编程对抓包数据进行自动化分析，提升网络运维和开发的效率。

5 演示

我们的脚本集成了自动访问网站，以防止用户手动访问网站，从而确保 TCP 三次握手过程的触发的准确性。首先运行脚本时，会提示用户输入要访问的网站 URL，然后脚本会自动访问该网站并抓取 TCP 数据包，此处以 **https://zh.wikipedia.org/wiki/Lean** 这个网站为例。

抓包完成后，脚本会解析抓取到的数据包，并输出 TCP 三次握手过程中的关键信息。

```
        Source Address: 172.24.32.1
        Destination Address: 172.24.41.234
Layer TCP
:       Source Port: 7890
        Destination Port: 58564
        Stream index: 0
        Conversation completeness: Incomplete, DATA (15)
        TCP Segment Len: 2776
        Sequence Number: 20357    (relative sequence number)
        Sequence Number (raw): 4192639337
        Next Sequence Number: 23133    (relative sequence number)
        Acknowledgment Number: 801    (relative ack number)
        Acknowledgment number (raw): 892263170
        1000 .... = Header Length: 32 bytes (8)
        Flags: 0x018 (PSH, ACK)
        000. .... .... = Reserved: Not set
        ...0 .... .... = Nonce: Not set
        .... 0... .... = Congestion Window Reduced (CWR): Not set
        .... .0.. .... = ECN-Echo: Not set
        .... ..0. .... = Urgent: Not set
        .... ...1 .... = Acknowledgment: Set
        .... .... 1... = Push: Set
        .... .... .0.. = Reset: Not set
        .... .... ..0. = Syn: Not set
        .... .... ...0 = Fin: Not set
        TCP Flags: ·******AP···
        Window: 8192
        Calculated window size: 2097152
        Window size scaling factor: 256
        Checksum: 0xad1a [unverified]
        Checksum Status: Unverified
        Urgent Pointer: 0
        Options: (12 bytes), No-Operation (NOP), No-Operation (NOP), Timestamps
        TCP Option - No-Operation (NOP)
        Kind: No-Operation (1)
        TCP Option - Timestamps: TSval 325254038, TSecr 3301048458
        Length: 10
        Timestamp value: 325254038
        Timestamp echo reply: 3301048458
        Timestamps
        Time since first frame in this TCP stream: 1.332342000 seconds
        Time since previous frame in this TCP stream: 0.000000000 seconds
        SEQ/ACK analysis
        iRTT: 0.000244000 seconds
        Bytes in flight: 5552
        Bytes sent since last PSH flag: 2776
        TCP payload (2776 bytes)
        TCP segment data (2776 bytes)
        TCP Option - No-Operation (NOP)
        Kind: No-Operation (1)
        Kind: Time Stamp Option (8)

2024-11-07 18:47:44,206 - INFO - Source IP: 172.24.32.1, Destination IP: 172.24.41.234
```

此处一页截图放不下，仅作展示，实际输出会包含更多详细信息。
此外，本脚本将抓取到的数据包保存至本地，存入 log 文件，以便后续进一步分析。

```
ComputerNetwork > CNLab > packet_capture.log
Log: Show Apex Log Analysis
2024-11-07 18:46:34,879 - INFO - Starting tcpdump to capture packets...
2024-11-07 18:46:48,960 - INFO - Starting tcpdump to capture packets...
2024-11-07 18:47:37,445 - INFO - Visiting website: https://zh.wikipedia.org/wiki/Lean
2024-11-07 18:47:38,793 - INFO - Website https://zh.wikipedia.org/wiki/Lean responded with status
2024-11-07 18:47:43,803 - INFO - Analyzing the captured packets from tcp_handshake.pcap...
2024-11-07 18:47:44,032 - INFO - Packet Info: Packet (Length: 74)
[1m[33mLayer ETH
:[0m[1m[32m Destination:[0m[1m 00:15:5d:e9:61:77
[0m[1m[32m  Address:[0m[1m 00:15:5d:e9:61:77
[0m[1m[32m  .... ..0. .... .... .... .... = LG bit:[0m[1m Globally unique address (factory default
[0m[1m[32m  .... ...0 .... .... .... .... = IG bit:[0m[1m Individual address (unicast)
[0m[1m[32m  Source:[0m[1m 00:15:5d:e9:62:09
[0m[1m[32m  .... ..0. .... .... .... .... = LG bit:[0m[1m Globally unique address (factory default
[0m[1m[32m  .... ...0 .... .... .... .... = IG bit:[0m[1m Individual address (unicast)
[0m[1m[32m  Type:[0m[1m IPv4 (0x0800)
[0m[1m[32m  Address:[0m[1m 00:15:5d:e9:62:09
[0m[1m[33mLayer IP
:[0m[1m[32m 0100 .... = Version:[0m[1m 4
[0m[1m[32m  .... 0101 = Header Length:[0m[1m 20 bytes (5)
[0m[1m[32m  Differentiated Services Field:[0m[1m 0x00 (DSCP: CS0, ECN: Not-ECT)
[0m[1m[32m  0000 00.. = Differentiated Services Codepoint:[0m[1m Default (0)
[0m[1m[32m  .... ..00 = Explicit Congestion Notification:[0m[1m Not ECN-Capable Transport (0)
[0m[1m[32m  Total Length:[0m[1m 60
[0m[1m[32m  Identification:[0m[1m 0x87b3 (34739)
[0m[1m[32m  Flags:[0m[1m 0x40, Don't fragment
[0m[1m[32m  0... .... = Reserved bit:[0m[1m Not set
[0m[1m[32m  .1.. .... = Don't fragment:[0m[1m Set
```

6 实验结果与分析

本实验的抓包与分析结果表明，TCP 三次握手过程如下：

- 客户端首先发送 SYN 包以请求建立连接，包含初始序列号。
- 服务器收到 SYN 包后，回复 SYN-ACK 包，包含服务器的初始序列号以及对客户端 SYN 包的确认。
- 客户端收到 SYN-ACK 包后，回复 ACK 包，确认与服务器建立连接。

通过 PyShark 对抓包文件的解析，我们能够详细观察到这些数据包的内容，包括 IP 地址、端口号、序列号等信息。

7 结论

本文档描述了如何通过 Wireshark 抓包和 Python 编程来捕获和分析 TCP 三次握手的过程。通过实验，详细展示了 TCP 连接的建立过程，并通过数据包的内容分析了其具体实现。该实验加深了对 TCP 协议的理解，尤其是在建立可靠连接方面的机制和细节。

8 未来工作

未来可以考虑以下改进：

- 实现对其他协议（如 UDP、HTTP）的抓包与分析，以更全面地理解网络通信。
- 优化抓包和分析的流程，以支持实时分析和更复杂的数据过滤条件。
- 使用图形化工具展示抓包结果，使分析结果更直观。

姓名: 李俊洁
班级: 计算机 2204
学号: 20225443
课程: 计算机网络
日期: November 6, 2024
制作: LaTeX
感谢您的批阅！

References

[1] tcpdump 官方文档：https://www.tcpdump.org/manpages/tcpdump.1.html

[2] Wireshark 用户指南：https://www.wireshark.org/docs/

[3] PyShark 文档：https://github.com/KimiNewt/pyshark